



POLITÉCNICA

Universidad Politécnica de Madrid

Bases de Datos II

Práctica Neo4j: Jotun's Lair

Consultas Cypher, Graph Data Science y recomendaciones de diseño

Yixuan Lu Guo
Shengkai Zhu

9 de mayo de 2026

Índice

1. Introducción	2
2. Entorno de ejecución	2
3. Modelo de datos	2
4. Consultas Cypher	3
4.1. Consulta 1: objetos necesarios sin forma directa de obtención	3
4.2. Consulta 2: recetas con mayor ganancia	4
4.3. Consulta 3: arma crafteable con mayor daño de fuego	4
4.4. Consulta 4: arma con mayor daño combinado por tipo	5
4.5. Consulta 5: proceso para obtener el arma 297	5
4.6. Consulta 6: monstruos necesarios para fabricar el arma 880	5
4.7. Consulta 7: proceso de fabricación más complejo	6
4.8. Consulta 8: arma que requiere matar más monstruos distintos	6
4.9. Consulta 9: debilidad máxima por localización	6
4.10. Consulta 10: recomendación de arma por localización	7
5. Análisis de grafos con Graph Data Science	8
5.1. Creación de relaciones SIMILAR	8
5.2. Proyección del grafo	8
5.3. Detección de comunidades con Leiden	9
5.4. PageRank por comunidad	9
6. Recomendaciones de diseño	10
7. Conclusiones	11

1. Introducción

Esta práctica se centra en el uso de **Neo4j** para modelar y consultar un sistema de craqueo de armas del videojuego ficticio *Jotun's Lair*. En este dominio, el jugador obtiene materiales cazando monstruos, crea armas directamente mediante procesos de craqueo y también puede mejorar armas mediante procesos de actualización.

El uso de una base de datos de grafos resulta adecuado porque el problema contiene muchas relaciones naturales entre entidades: armas, monstruos, materiales, recetas, localizaciones y elementos. Además, las armas no solo tienen daño base, sino también posibles daños elementales, mientras que los monstruos pueden ser débiles o resistentes a diferentes elementos.

La práctica se divide en tres partes principales:

- Realización de consultas Cypher sobre el grafo.
- Análisis de grafos con *Graph Data Science* para agrupar armas similares.
- Reflexión de diseño sobre si conviene modelar los tipos de armas como nodos.

2. Entorno de ejecución

La base de datos se ha ejecutado mediante Docker, usando Neo4j 5.25 y los plugins **APOC** y **Graph Data Science**. Para cargar el dump inicial de la base de datos se utiliza el siguiente comando:

```
docker run -it --rm \
  --volume="$PWD/data:/data" \
  --volume="$PWD/import:/backups" \
  neo4j:5.25 \
  neo4j-admin database load neo4j --from-path=/backups --overwrite-
  destination=true --verbose
```

Después, Neo4j se arranca con:

```
docker run --rm \
  --name neo4j-jotuns \
  --volume="$PWD/data:/data" \
  --volume="$PWD/plugins:/plugins" \
  --publish=7474:7474 \
  --publish=7687:7687 \
  --env NEO4J_AUTH=neo4j/password \
  --env NEO4J_PLUGINS='["apoc","graph-data-science"]' \
  neo4j:5.25
```

La interfaz web queda disponible en:

<http://localhost:7474>

con usuario **neo4j** y contraseña **password**.

3. Modelo de datos

El grafo está formado principalmente por los siguientes tipos de nodos:

- **Weapon**: armas del juego.
- **WeaponCraft**: proceso de creación directa de un arma.

- **WeaponUpgrade**: proceso de mejora o actualización de un arma.
- **Item**: materiales u objetos.
- **Recipe**: recetas para producir items.
- **Monster**: monstruos que pueden dropear materiales.
- **Location**: zonas o biomas donde viven los monstruos.
- **Element**: elementos de daño o estado.

Las relaciones principales del modelo son:

- **PRODUCES**: indica que una receta o proceso produce un item o arma.
- **NEEDS**: indica qué materiales necesita una receta, craftero o mejora.
- **TRANSFORMS**: indica que una mejora transforma un arma en otra.
- **DROPS**: indica qué materiales puede soltar un monstruo.
- **LIVES_IN**: indica en qué localización vive un monstruo.
- **WEAK_TO**: indica la debilidad de un monstruo frente a un elemento.
- **RESISTENT_TO**: indica la resistencia de un monstruo frente a un elemento.
- **DEALS**: indica el daño elemental extra que causa un arma.

4. Consultas Cypher

En esta sección se explican las diez consultas Cypher desarrolladas sobre la base de datos. Cada consulta responde a una pregunta concreta del enunciado.

4.1. Consulta 1: objetos necesarios sin forma directa de obtención

```
MATCH (i:Item)
WHERE NOT EXISTS { (:Recipe)-[:PRODUCES]->(i) }
AND NOT EXISTS { (:Monster)-[:DROPS]->(i) }
AND (
  EXISTS { (:WeaponCraft)-[:NEEDS]->(i) }
  OR EXISTS { (:WeaponUpgrade)-[:NEEDS]->(i) }
)
RETURN i.name AS nombre, i.description AS descripcion
```

Esta consulta busca objetos que se utilizan para crear o mejorar armas, pero que no se pueden obtener ni mediante recetas ni matando monstruos. Para ello se parte de los nodos **Item** y se descartan aquellos que aparecen como salida de una receta o como drop de un monstruo. Después se comprueba que el item sí sea necesario en algún **WeaponCraft** o **WeaponUpgrade**.

4.2. Consulta 2: recetas con mayor ganancia

```
MATCH (r:Recipe)-[p:PRODUCES]->(output:Item)
MATCH (r)-[:NEEDS]->(input:Item)

WITH r,
     collect(input) AS inputs,
     output,
     p.amount AS outAmount

WHERE ALL(i IN inputs WHERE NOT EXISTS { (:Recipe)-[:PRODUCES]->(i) })

WITH r,
     output,
     outAmount,
     inputs,
     reduce(
         coste = 0,
         i IN inputs | coste + coalesce(i.value, 0)
     ) AS costeTotal

WITH r,
     [i IN inputs | i.name] AS itemsInput,
     output.name AS itemOutput,
     coalesce(output.value, 0) * coalesce(outAmount, 1) - costeTotal AS
     ganancia

RETURN r.id AS recetaId,
       itemsInput,
       itemOutput,
       ganancia
ORDER BY ganancia DESC
LIMIT 5
```

Esta consulta calcula las cinco recetas más rentables. Primero obtiene el ítem generado y los ítems necesarios. Después, mediante ALL, se filtran las recetas cuyos inputs no sean producidos por otras recetas. El coste total se calcula con reduce, sumando los valores de los inputs. La ganancia se obtiene restando ese coste al valor total producido.

4.3. Consulta 3: arma crafteable con mayor daño de fuego

```
MATCH (:Element {name:'fire'})<-[d:DEALS]-(w:Weapon)<-[:PRODUCES]-(wc:
    WeaponCraft)

WITH w, wc, d
ORDER BY d.damage DESC
LIMIT 1

MATCH (wc)-[:NEEDS]->(i:Item)
OPTIONAL MATCH (m:Monster)-[:DROPS]->(i)

WITH w,
     collect(DISTINCT m.name) AS monstersRaw,
     collect(DISTINCT CASE WHEN m IS NULL THEN i.name END) AS extrasRaw

RETURN w.name AS nombre,
       w.kind AS tipo,
```

```
[m IN monstersRaw WHERE m IS NOT NULL] AS monstruos,
[e IN extrasRaw WHERE e IS NOT NULL] AS materialesExtra
```

Primero se localizan las armas que hacen daño de fuego y que se pueden crear directamente. Se ordenan por daño de fuego y se selecciona la mejor. Después se revisan sus materiales necesarios y se comprueba qué monstruos los dropean. Los materiales que no provienen de ningún monstruo se devuelven como materiales extra.

4.4. Consulta 4: arma con mayor daño combinado por tipo

```
MATCH (w:Weapon)
OPTIONAL MATCH (w)-[d:DEALS]->(:Element {kind: 'element'})

WITH w,
      coalesce(w.damage, 0) + coalesce(sum(d.damage), 0) AS danoTotal

WITH w.kind AS tipo,
      max(danoTotal) AS maxDano,
      collect({nombre: w.name, dano: danoTotal}) AS armas

RETURN tipo,
        [a IN armas WHERE a.dano = maxDano | a.nombre] AS armas,
        maxDano AS danoCombinado

ORDER BY danoCombinado DESC
```

Esta consulta calcula el daño total de cada arma como la suma del daño base y todos los daños extra de tipo elemental. Se usa OPTIONAL MATCH para incluir también armas sin daño elemental. Después se agrupa por tipo de arma y se devuelven las armas que alcanzan el daño máximo dentro de cada tipo, teniendo en cuenta posibles empates.

4.5. Consulta 5: proceso para obtener el arma 297

```
MATCH path = (w2:Weapon {id:297})<-[:TRANSFORMS*]-(w1:Weapon)
WHERE EXISTS { (w1)<-[:PRODUCES]-(:WeaponCraft) }

WITH [n IN nodes(path) WHERE n:Weapon | n.id] AS weapons

RETURN weapons AS proceso
```

La consulta reconstruye el proceso de actualización del arma con id 297. Se busca un camino de relaciones TRANSFORMS desde un arma base hasta el arma objetivo. La condición EXISTS asegura que el arma inicial del camino sea crafteable directamente. Finalmente se devuelven los identificadores de las armas del proceso.

4.6. Consulta 6: monstruos necesarios para fabricar el arma 880

```
MATCH path = (target:Weapon {id: 880})<-[:TRANSFORMS*]-(base:Weapon)<-[:
  PRODUCES]-(wc:WeaponCraft)

UNWIND [n IN nodes(path) WHERE n:WeaponCraft OR n:WeaponUpgrade] AS step

MATCH (step)-[:NEEDS]->(:Item)<-[:DROPS]-(m:Monster)

RETURN DISTINCT m.name AS monstruo
ORDER BY monstruo
```

Esta consulta busca la cadena de fabricación del arma 880 y extrae los pasos que necesitan materiales, es decir, `WeaponCraft` y `WeaponUpgrade`. Con `UNWIND` se procesa cada paso por separado. Después se buscan los monstruos que dropean los items necesarios y se devuelven sin duplicados.

4.7. Consulta 7: proceso de fabricación más complejo

```
MATCH path = (base:Weapon) -[:TRANSFORMS*] -> (target:Weapon)
WHERE EXISTS { (:WeaponCraft) -[:PRODUCES] -> (base) }
AND NOT EXISTS { (:WeaponCraft) -[:PRODUCES] -> (target) }

WITH target,
     [n IN nodes(path) WHERE n:Weapon | n.name] AS armas,
     size([n IN nodes(path) WHERE n:Weapon]) - 1 AS numero_de_pasos

ORDER BY numero_de_pasos DESC
LIMIT 1

RETURN target.name AS arma,
       target.kind AS tipo,
       armas AS armas_intermedias,
       numero_de_pasos
```

Se buscan caminos de actualización desde armas crafteables hasta armas objetivo. El número de pasos se calcula contando las transiciones entre armas. Después se ordenan los caminos de mayor a menor longitud y se selecciona el más largo, que representa el proceso de fabricación más complejo.

4.8. Consulta 8: arma que requiere matar más monstruos distintos

```
MATCH path = (wc:WeaponCraft) -[:PRODUCES] -> (base:Weapon) -[:TRANSFORMS
*0..] -> (target:Weapon)

UNWIND [n IN nodes(path) WHERE n:WeaponCraft OR n:WeaponUpgrade] AS step

MATCH (step) -[:NEEDS] -> (:Item) <-[:DROPS] - (m:Monster)

WITH target.name AS nombre,
     [n IN nodes(path) WHERE n:Weapon | n.name] AS armas,
     collect(DISTINCT m.name) AS monstruos

RETURN nombre,
       armas,
       monstruos,
       size(monstruos) AS numero_monstruos

ORDER BY numero_monstruos DESC
LIMIT 1
```

La consulta analiza todos los procesos de fabricación posibles, incluyendo armas fabricadas directamente gracias a `*0..`. Para cada proceso, obtiene los monstruos que dropean los materiales necesarios y cuenta cuántos monstruos distintos intervienen. Se devuelve el proceso que requiere matar más monstruos diferentes.

4.9. Consulta 9: debilidad máxima por localización

```

MATCH (l:Location)<-[:LIVES_IN]-(m:Monster)-[w:WEAK_TO]->(e:Element {
  kind: 'element'})

WITH l,
     e.name AS elemento,
     sum(w.level) AS debilidadTotal

WITH l,
     collect({elemento: elemento, debilidad: debilidadTotal}) AS
     debilidades,
     max(debilidadTotal) AS maxDebilidad

RETURN l.name AS ubicacion,
        maxDebilidad AS debilidadMaxima,
        [d IN debilidades WHERE d.debilidad = maxDebilidad | d.elemento]
        AS elementos

ORDER BY ubicacion

```

Para cada localización se buscan los monstruos que viven allí y sus debilidades elementales. Se suman los niveles de debilidad por localización y elemento. Después se calcula la debilidad máxima de cada zona y se devuelven los elementos que alcanzan ese valor, permitiendo empates.

4.10. Consulta 10: recomendación de arma por localización

```

MATCH (l:Location)<-[:LIVES_IN]-(m:Monster)-[weak:WEAK_TO]->(e:Element {
  kind: 'element'})

WITH l,
     e,
     sum(weak.level) AS debilidadTotal

WITH l,
     collect({
       elemento: e,
       nombre: e.name,
       debilidad: debilidadTotal
     }) AS debilidades,
     max(debilidadTotal) AS maxDebilidad

WITH l,
     [d IN debilidades WHERE d.debilidad = maxDebilidad | d.elemento] AS
     elementosVulnerables,
     [d IN debilidades WHERE d.debilidad = maxDebilidad | d.nombre] AS
     nombresElementos

MATCH (arma:Weapon {kind: $tipoArma})
OPTIONAL MATCH (arma)-[deal:DEALS]->(e:Element {kind: 'element'})

WITH l,
     nombresElementos,
     arma,
     sum(
       CASE
         WHEN e IN elementosVulnerables THEN coalesce(deal.damage, 0)
         ELSE 0
       END

```



```

    ) AS danoContraElementos

WITH l,
     nombresElementos,
     collect({arma: arma.name, dano: danoContraElementos}) AS armas,
     max(danoContraElementos) AS maxDano

RETURN l.name AS ubicacion,
       nombresElementos AS elementosVulnerables,
       [a IN armas WHERE a.dano = maxDano | a.arma] AS armasRecomendadas

ORDER BY ubicacion

```

Esta consulta recomienda armas de un tipo indicado por parámetro. Primero calcula los elementos frente a los que los monstruos de cada localización son más vulnerables. Después compara las armas del tipo elegido y suma solo el daño que hacen contra esos elementos vulnerables. Finalmente devuelve las armas con mayor daño útil para cada localización.

5. Análisis de grafos con Graph Data Science

La segunda parte de la práctica utiliza el cliente Python de *Graph Data Science*. El objetivo es agrupar armas según sus materiales de fabricación o actualización.

5.1. Creación de relaciones SIMILAR

Primero se crea una relación **SIMILAR** entre dos armas si comparten al menos un material. El peso de la relación se guarda en la propiedad **weight** y se calcula con el índice de Jaccard:

$$Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

donde A y B son los conjuntos de materiales necesarios para construir cada arma.

En el notebook se obtuvieron los siguientes resultados:

relacionesCreadas	pesoMinimo	pesoMaximo	pesoMedio
28140	0.125	1.0	0.322835

Estos valores indican que se crearon muchas relaciones de similitud. El peso máximo 1.0 significa que existen pares de armas que comparten exactamente el mismo conjunto de materiales.

También se realizaron comprobaciones para confirmar que no había pesos inválidos ni relaciones de un arma consigo misma.

5.2. Proyección del grafo

Para aplicar algoritmos GDS, se proyecta un grafo en memoria con nodos **Weapon** y relaciones **SIMILAR**. La relación se proyecta como no dirigida:

```

G, resultado_proyeccion = gds.graph.project(
    nombre_grafo,
    "Weapon",
    {
        "SIMILAR": {
            "orientation": "UNDIRECTED",
            "properties": "weight"
        }
    }
)

```

)

Esto es importante porque la similitud no tiene dirección real: si un arma A es similar a un arma B, B también es similar a A.

En la proyección se obtuvieron:

Nodos	Relaciones
1024	56280

Aunque en Neo4j se crearon 28140 relaciones **SIMILAR**, GDS muestra 56280 al proyectarlas como no dirigidas, ya que internamente considera ambos sentidos.

5.3. Detección de comunidades con Leiden

Después se aplica el algoritmo Leiden:

```
resultado_leiden = gds.leiden.write(  
    G,  
    writeProperty="comunidad",  
    relationshipWeightProperty="weight",  
    randomSeed=19  
)
```

Leiden agrupa las armas en comunidades según sus relaciones de similitud. La propiedad **weight** hace que las armas con mayor similitud tengan más influencia al formar comunidades. El resultado se guarda en cada nodo **Weapon** con la propiedad **comunidad**.

Los resultados principales fueron:

Propiedad	Valor
Nodos escritos	1024
Comunidades detectadas	58
Convergencia	True
Modularidad	0.790631

La modularidad obtenida es alta, por lo que las comunidades están razonablemente bien separadas.

5.4. PageRank por comunidad

Por último, se aplica PageRank dentro de cada comunidad. Para ello se crea un subgrafo por comunidad mediante **gds.graph.filter**. Este método es más limpio que duplicar relaciones manualmente, porque el grafo ya se proyectó como no dirigido.

```
query_filtrar_subgrafo = """  
CALL gds.graph.filter(  
    $subgraphName,  
    $graphName,  
    $nodeFilter,  
    '*'  
)  
YIELD graphName, nodeCount, relationshipCount  
RETURN graphName, nodeCount, relationshipCount  
"""
```

El filtro utilizado es de la forma:

```
nodeFilter = f"n.comunidad = {comunidad}"
```

Así, cada subgrafo contiene solo las armas de una comunidad concreta. Después se ejecuta PageRank usando `weight` como peso:

```
df_pagerank = gds.pageRank.stream(  
    Gsub,  
    relationshipWeightProperty="weight"  
)
```

Las armas con mayor PageRank dentro de cada comunidad se consideran representantes de dicha comunidad.

Los resultados muestran comunidades de tamaños distintos. Algunas comunidades son grandes y muy conectadas, mientras que otras tienen un solo nodo y cero relaciones. Esto no es un error: simplemente representa armas aisladas que no comparten suficientes materiales con otras.

6. Recomendaciones de diseño

La última parte analiza si tiene sentido cambiar el atributo `kind` de `Weapon` a un nodo independiente `WeaponKind`.

El diseño actual representa el tipo como propiedad:

```
(:Weapon {kind: "sword"})
```

El diseño alternativo sería:

```
(:Weapon) -[:HAS_KIND] -> (:WeaponKind {name: "sword"})
```

La comprobación realizada muestra que los valores de `kind` son categorías planas, como:

- bow
- charge-blade
- dual-blades
- great-sword
- hammer
- lance
- sword-shield

En el estado actual del modelo, `kind` funciona como una propiedad descriptiva. No se han detectado subtipos, jerarquías ni relaciones propias asociadas a los tipos de arma. Por tanto, mantenerlo como propiedad es suficiente.

No obstante, tendría sentido crear nodos `WeaponKind` si en el futuro los tipos de arma tuvieran atributos propios, relaciones con otros nodos, jerarquías de subtipos o se usaran como punto de entrada frecuente en consultas.

7. Conclusiones

La práctica ha permitido aplicar Neo4j a un dominio con relaciones complejas, como es el sistema de craftero y mejora de armas de *Jotun's Lair*. Las consultas Cypher permiten navegar de forma natural por cadenas de fabricación, materiales, monstruos, elementos y localizaciones.

Además, el uso de Graph Data Science permite ir más allá de las consultas tradicionales. Mediante el índice de Jaccard se construye una red de similitud entre armas, Leiden permite detectar comunidades de armas parecidas y PageRank ayuda a seleccionar representantes dentro de cada comunidad.

Finalmente, la parte de diseño muestra que no todos los atributos deben convertirse automáticamente en nodos. En este caso, el tipo de arma puede mantenerse como propiedad mientras siga siendo una categoría simple, aunque el modelo podría evolucionar si los tipos adquirieran más importancia en el dominio.